PropOS — 开发环境搭建指南

> 版本: v1.0
>
> 日期: 2026-04-08
>
> 适用系统: macOS（主力开发机）
>
> 验证脚本: `scripts/check_env.sh`

目录

1. 概述
2. 后端工具链
   - 2.1 Dart SDK（需 ≥ 3.0）
   - 2.2 PostgreSQL（需 ≥ 15）
3. 前端工具链
   - 3.1 Node.js（需 ≥ 18，推荐 20 LTS）
   - 3.2 前端项目依赖安装
4. Python 工具链
   - 4.1 Python 3
   - 4.2 创建项目虚拟环境（必需）
   - 4.3 python-docx（必需）
   - 4.4 ezdxf（M1 模块，可选）
5. macOS 专属工具
   - 5.1 Pages.app（必需）
   - 5.2 ODA File Converter（M1 模块，可选）
6. 通用工具
   - 6.1 Git
7. 后端环境变量配置
   - 7.1 创建`.env`文件
   - 7.2 必需变量说明
   - 7.3 可选变量
   - 7.4 生成随机密钥
   - 7.5 创建文件存储目录
8. 数据库初始化
   - 8.1 创建用户和数据库
   - 8.2 验证连接

8.3 运行数据库迁移
9. 项目脚手架初始化
9.1 后端（Dart Shelf）
9.2 前端（Flutter + admin）
10. 环境验证
11. 常见问题
Q: `dart: command not found`
Q: `psql: error: connection refused`
Q: `psql: command not found`（PostgreSQL 安装后找不到命令）
Q: `dart pub get` 报网络超时
Q: `npm install` 报网络超时
Q: `Pages` 导出 PDF 失败 / 脚本卡住
Q: `ENCRYPTION_KEY` 应该是什么格式?

---

1. 概述

PropOS 后端使用 **Dart + Shelf**，移动端使用 **Flutter（Dart）**，PC 后台使用 **Vue 3 + Element Plus + Vite**，数据库使用 **PostgreSQL 15+**。

文档转换流水线依赖 Python + macOS Pages.app；CAD 图纸导入（M1 模块）还需要 ODA File Converter + ezdxf。所有工具安装完成后，运行以下命令验证：

```
bash scripts/check_env.sh
```

输出全绿（0 项失败）即表示环境就绪。

---

2. 后端工具链

2.1 Dart SDK（需 ≥ 3.0）

推荐单独安装 Dart SDK，前端使用 Node.js。

安装 **Homebrew**（若未安装）：

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

通过 **Homebrew** 安装：

```
brew tap dart-lang/dart
brew install dart
```

```
**验证：**

```bash
dart --version
# 期望输出：Dart SDK version: 3.x.x ...
```
```

2.2 PostgreSQL（需 ≥ 15）

```
# 安装 PostgreSQL 15
brew install postgresql@15

# 将命令行工具加入 PATH（写入 shell 配置文件）
echo 'export PATH="/opt/homebrew/opt/postgresql@15/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc

# 启动服务（开机自启）
brew services start postgresql@15

# 验证
psql --version
# 期望输出：psql (PostgreSQL) 15.x
```

> 注意：Apple Silicon Mac 的 Homebrew 路径为 `/opt/homebrew`，Intel Mac 为 `/usr/local`，请按实际情况调整 PATH。

---

3. 前端工具链

3.1 Node.js（需 ≥ 18，推荐 20 LTS）

通过 **nvm** 安装（推荐，便于版本管理）：

```
# 安装 nvm
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.zshrc

# 安装 Node.js 20 LTS
nvm install 20
nvm use 20
nvm alias default 20
```

或通过 **Homebrew**（版本固定）：

```
brew install node@20
```

验证：

```
node --version
# 期望输出：v20.x.x
```

```
npm --version
# 期望输出：10.x.x
```

3.2 前端项目依赖安装

```
# Flutter 端
cd /Users/wanxt/app/propos/flutter_app
flutter pub get

# admin 端
cd /Users/wanxt/app/propos/admin
npm install
```

> Flutter 端使用 `pubspec.yaml` 管理依赖，admin 端使用 `package.json` 管理依赖。

---

4. Python 工具链

Python 工具链用于文档转换流水线（Markdown → Word → PDF）和 CAD 图纸处理。

4.1 Python 3

macOS 系统自带 Python 3，通常无需额外安装。

```
python3 --version
# 期望输出：Python 3.x.x
```

如版本过旧或未安装：

```
brew install python@3.12
```

4.2 创建项目虚拟环境（必需）

Python 3.13+ 默认禁止 `pip3` 直接向系统环境安装包（PEP 668）。

项目统一使用项目根目录下的 `.venv` 虚拟环境，所有脚本优先使用 `.venv/bin/python3` 。

```
# 在项目根目录创建虚拟环境（仅需执行一次）
cd /path/to/propos
python3 -m venv .venv
```

> `.venv/` 已加入 `.gitignore` ，不会提交到版本库。

4.3 python-docx（必需）

文档转换脚本 `scripts/md2word.py` 依赖此包。

```
# 安装到 .venv（无需激活，直接使用 .venv 的 pip）
.venv/bin/pip install python-docx
```

```
# 验证
.venv/bin/python3 -c "import docx; print(docx.__version__)"
```

4.4 ezdxf（M1 模块，可选）

M1 资产模块的 CAD 图纸导入功能需要（DXF → SVG 转换）。

```
.venv/bin/pip install "ezdxf[draw]"

# 验证
.venv/bin/python3 -c "import ezdxf; print(ezdxf.__version__)"
```

> 如果不涉及 CAD 图纸导入功能，此包可暂缓安装，不影响其他模块开发。

一次性安装两个包：

```
.venv/bin/pip install python-docx "ezdxf[draw]"
```

---

5. macOS 专属工具

5.1 Pages.app（必需）

`scripts/docx2pdf.py` 通过 AppleScript 驱动 Pages 将 Word 文档导出为 PDF。macOS 系统通常已预装，如未安装：

1. 打开 **App Store**
2. 搜索 **Pages**
3. 点击安装（免费）

验证：

```
ls /Applications/Pages.app
# 存在即为已安装
```

5.2 ODA File Converter（M1 模块，可选）

用于 CAD 图纸导入的第一步：将 `.dwg` 文件转换为 `.dxf` 格式。

下载页：https://www.opendesign.com/guestfiles/oda*file*converter

根据芯片类型选择对应版本（当前 v27.1）：

芯片	直接下载
Apple Silicon（M 系列）	ODAFileConverter_QT6_macOsX_arm64_15.0dll_27.1.dmg
Intel	ODAFileConverter_QT6_macOsX_x64_15.0dll_27.1.dmg

安装：打开 `.dmg` ，拖拽到 `/Applications/`

验证：

```
ls /Applications/ODAFileConverter.app
# 存在即为已安装
```

> 仅开发 M1（资产与空间可视化）模块时需要，其他模块可跳过。

---

6. 通用工具

6.1 Git

macOS 通常已预装。如未安装或版本过旧：

```
brew install git

# 配置用户信息（首次使用）
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

验证：

```
git --version
```

---

7. 后端环境变量配置

后端服务通过环境变量读取所有运行时配置。缺少任何必需变量时，服务将拒绝启动。

7.1 创建 `.env` 文件

在 `backend/` 目录下创建 `.env` 文件（已加入 `.gitignore` ，不提交到版本库）：

```
cp backend/.env.example backend/.env   # 如果存在模板
# 或直接创建：
touch backend/.env
```

7.2 必需变量说明

编辑 `backend/.env` ，填入以下变量（所有变量均不得为空）：

```
# ─── 数据库 ──────────────────────────────────
# PostgreSQL 连接串，格式：postgres://用户名:密码@主机:端口/数据库名
DATABASE_URL=postgres://propos:your_password@localhost:5432/propos_dev

# ─── JWT 认证 ────────────────────────────────
# 签名密钥，长度 ≥ 32 位，请使用随机字符串
JWT_SECRET=请替换为32位以上的随机字符串
```

```
# Token 有效期，单位：小时
JWT_EXPIRES_IN_HOURS=24

# ─── 文件存储 ──────────────────────────────────
# 本地文件存储根目录（合同 PDF、工单照片等），需提前创建
FILE_STORAGE_PATH=/data/uploads

# ─── 加密 ──────────────────────────────────────
# AES-256 密钥，用于证件号加密，必须为 32 字节十六进制字符串（64 个十六进制字符）
ENCRYPTION_KEY=请替换为64位十六进制字符串

# ─── 服务端口 ──────────────────────────────────
APP_PORT=8080
```

7.3 可选变量

缺失时使用默认值，不阻断启动：

```
# 允许跨域请求的来源（多个用逗号分隔）
CORS_ORIGINS=http://localhost:3000,http://localhost:8080

# 日志级别：debug / info / warning / error
LOG_LEVEL=info

# 文件上传最大大小（MB）
MAX_UPLOAD_SIZE_MB=50
```

7.4 生成随机密钥

```
# 生成 JWT_SECRET（随机 40 位字符串）
openssl rand -base64 40

# 生成 ENCRYPTION_KEY（32 字节 = 64 位十六进制）
openssl rand -hex 32
```

7.5 创建文件存储目录

```
sudo mkdir -p /data/uploads
sudo chown $(whoami) /data/uploads
```

---

8. 数据库初始化

8.1 创建用户和数据库

```
# 进入 PostgreSQL 交互终端（使用系统用户认证）
psql postgres
```

```
# 在 psql 终端中执行:
CREATE USER propos WITH PASSWORD 'your_password';
CREATE DATABASE propos_dev OWNER propos;
GRANT ALL PRIVILEGES ON DATABASE propos_dev TO propos;
\q
```

> 将 `your_password` 替换为实际密码，并与 `DATABASE_URL` 中的密码保持一致。

8.2 验证连接

```
psql postgres://propos:your_password@localhost:5432/propos_dev -c "SELECT version();"
# 成功输出 PostgreSQL 版本信息即表示连接正常
```

8.3 运行数据库迁移

待后端脚手架初始化完成后（见第 9 节），执行迁移：

```
cd backend
dart run bin/migrate.dart
```

---

9. 项目脚手架初始化

9.1 后端（Dart Shelf）

```
cd /Users/wanxt/app/propos/backend

# 初始化 Dart Shelf 项目
dart create -t server-shelf . --force

# 安装依赖
dart pub get
```

9.2 前端（Flutter + admin）

```
# Flutter 端
cd /Users/wanxt/app/propos/flutter_app
flutter pub get

# admin 端
cd /Users/wanxt/app/propos/admin
npm install
```

---

10. 环境验证

所有步骤完成后，运行环境检查脚本：

```
cd /Users/wanxt/app/propos
bash scripts/check_env.sh
```

预期输出示例（全部就绪）：

```
▸ 1. 后端工具链
  ✓  Dart SDK 3.x.x   (需 ≥3.0)
  ✓  PostgreSQL 15.x   (需 ≥15)

▸ 2. 前端工具链
  ✓  Node.js 20.x   (需 ≥18)
  ✓  npm 10.x

▸ 3. Python 工具链
  ✓  Python 3.x
  ✓  python-docx 1.x.x   (md→docx 转换)
  ✓  ezdxf x.x.x   (CAD DXF→SVG 转换)

▸ 4. macOS 专属工具
  ✓  Pages.app   (docx→PDF 导出)
  ✓  ODA File Converter   (DWG→DXF, CAD 转换第一步)

▸ 5. 通用工具
  ✓  Git 2.x.x
  ✓  Bash 5.x

▸ 6. 后端必需环境变量
  ✓  DATABASE_URL  — PostgreSQL 连接串
  ✓  JWT_SECRET  — JWT 签名密钥
  ✓  JWT_EXPIRES_IN_HOURS  — Token 有效期(小时)
  ✓  FILE_STORAGE_PATH  — 本地文件存储根目录
  ✓  ENCRYPTION_KEY  — AES-256 证件号加密密钥
  ✓  APP_PORT  — HTTP 监听端口

▸ 7. 数据库连通性
  ✓  数据库 propos_dev@localhost:5432  连通正常

▸ 8. 项目目录结构
  ✓  backend/  — 后端 Dart 项目
  ✓  flutter_app/  — Flutter 移动端项目
  ✓  admin/  — PC 后台 Vue 3 项目
  ✓  docs/  — 项目文档
  ✓  scripts/  — 工具脚本
  ✓  pdfdocs/  — PDF 文档输出
```

```
  ✓  backend/pubspec.yaml  — Dart 项目已初始化
  ✓  flutter_app/pubspec.yaml  — Flutter 项目已初始化
  ✓  admin/package.json  — admin 项目已初始化

✅ 环境完全就绪，可开始开发！
```

快速模式（跳过详细检查）：

```
bash scripts/check_env.sh --quick
```

脚本退出码等于失败项数，可在 CI 中使用：

```
bash scripts/check_env.sh --quick && echo "环境就绪" || echo "环境检查失败"
```

---

11. 常见问题

Q: `dart: command not found`

Dart SDK 未加入 PATH，通过 Homebrew 安装后手动确认：

```
# 如果通过 Homebrew 安装
brew info dart-lang/dart/dart

# 确认 PATH（Apple Silicon Mac）
echo 'export PATH="/opt/homebrew/opt/dart/libexec/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

Q: `psql: error: connection refused`

PostgreSQL 服务未启动：

```
brew services start postgresql@15
# 查看服务状态
brew services list | grep postgresql
```

Q: `psql: command not found`（PostgreSQL 安装后找不到命令）

Homebrew 安装的 PostgreSQL 15 不会自动链接到全局 PATH：

```
echo 'export PATH="/opt/homebrew/opt/postgresql@15/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

Q: `dart pub get` 报网络超时

配置 pub 代理或使用国内镜像：

```
export PUB_HOSTED_URL=https://pub.flutter-io.cn
dart pub get
```

也可将上述行写入 `~/.zshrc` 永久生效。

Q: `npm install` 报网络超时

使用国内 npm 镜像：

```
npm config set registry https://registry.npmmirror.com
npm install
```

Q: `Pages` 导出 PDF 失败 / 脚本卡住

Pages 首次打开某些文档可能弹出权限对话框，需要手动确认一次。之后使用 AppleScript 自动化即可正常工作。

Q: `ENCRYPTION_KEY` 应该是什么格式?

必须是 **64 位十六进制字符串**（即 32 字节），代表一个 AES-256 密钥：

```
# 生成符合要求的密钥
openssl rand -hex 32
# 输出示例：a3f1e2d4c5b6a7980911aabbccddeeff00112233445566778899 00aabbccddee
```